

ClaimScope: 面向科研想法生成前的主张、 假设与负面证据发现智能体

自然语言处理课程大作业实验报告

姓名: 刘子谦 学号: 23354118

项目仓库: <https://github.com/yueqian-coder/multi>

目录

摘要

1 背景与意义

2 需求分析与目标

3 系统设计

4 实现过程记录

5 实验设计

6 结果分析与反思

7 总结

附录 A 复现命令

附录 B 提交检查

摘要

科研工具通常从“搜索关键词”或“回答问题”开始，但研究者提出新想法之前更需要知道：方向中的核心说法是什么，哪些隐含假设已经获得证据，哪些仍然悬而未决，哪些负结果和边界条件值得绕开。本文设计并实现 ClaimScope，一个面向 idea 产生前阶段的 assumption-centric 多智能体系统。系统把模糊研究方向依次转换为可证伪核心主张、主张变体、隐含假设、四类证据查询、可追溯证据卡片以及有边界的实验机会点。核心主张模块采用三个 proposer、两个 critic 和一个 judge 的对抗式结构；最终 Web 界面要求六个角色都返回有效公开产物，任一阶段失败都会明确终止，不用启发式结果伪装成成功运行。系统同时提供中英双语 Streamlit Web 界面、命令行、Markdown 导出和五个 MCP 工具。最终在 27 个跨领域内部 ClaimBench 样例上得到 70.09/100 的启发式回归分数，并通过 111 项自动化测试。该分数只反映结构完整性与回归稳定性，不代表科学真实性。

关键词：科研智能体；多智能体；科学主张验证；负面证据；MCP；检索增强生成

1 背景与意义

从模糊方向到可执行研究问题之间存在一个常被忽略的中间层。研究者可能说“RAG 能减少幻觉”，但这句话没有说明比较基线、数据集、评价指标、适用边界与失败条件。直接搜索会带来大量相关论文，却不一定告诉用户：主张由哪些可检验假设组成，某篇论文究竟支持、限制还是直接反驳哪一项假设。

ClaimScope 将任务定义为“idea 产生之前的研究空间校准”。它不是论文库问答，也不替代系统综述；它的目标是把模糊方向转化为可以被验证、被反驳、被缩小范围的公开结构化产物。与一般 RAG 问答相比，本系统的差异在于以 assumption ledger 为中心组织证据，并主动检索 contradiction、limitation 与 null result，而不是只寻找支持材料。

相关工作为本系统提供了三个基础。RAG 说明外部检索可参与知识密集型生成；ReAct 说明语言模型可以交替执行推理与工具动作；SciFact 把科学主张验证形式化为证据检索及支持/反驳判断。PaperQA2 进一步显示语言智能体可以执行真实文献检索、综合与矛盾发现。ClaimScope 不与这些系统竞争完整答案质量，而是聚焦它们之前的“主张操作化与假设审计”缺口。

2 需求分析与目标

系统需满足六项功能目标：第一，输入可以是任意领域的模糊研究方向；第二，输出核心主张必须显式包含机制、目标、效果和可证伪测试；第三，把主张拆为可单独核验的隐含假设；第四，为每项假设生成支持、反驳、限制和零结果四类查询；第五，证据必须带来来源并把 unknown 与 unsupported 区分；第六，最终机会点必须足够小，可转化为含数据集、基线和指标的实验。

非功能目标包括：CLI 可无密钥复现确定性基线；Web 只运行真实 Agent 且失败时不伪造结果；不公开私有思维链；不记录 API 密钥；Web、CLI 和 MCP 共用同一服务边界；自动化测试覆盖正常、异常、严格失败与安装态场景。

3 系统设计

3.1 总体 workflow

ClaimScope: assumption-centric pre-ideation workflow

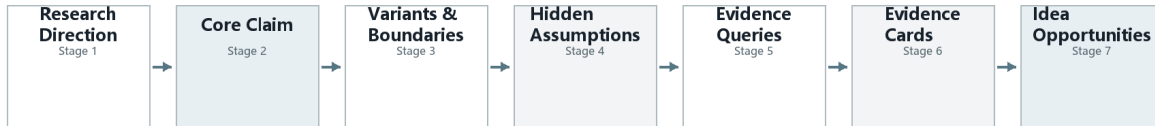


图 1 ClaimScope workflow

完整流程为 Research Direction → Core Claim → Claim Variants / Boundary Conditions → Hidden Assumptions → Evidence Queries → Evidence Cards → Idea Opportunities。每一步都生成 typed artifact，并记录角色、状态、耗时、公开摘要、产物数量与结构分数。Trace 展示的是可审计事件，而不是模型的私有 chain-of-thought。

3.2 核心主张多智能体竞技场

Core Claim Arena: public artifacts, adversarial review

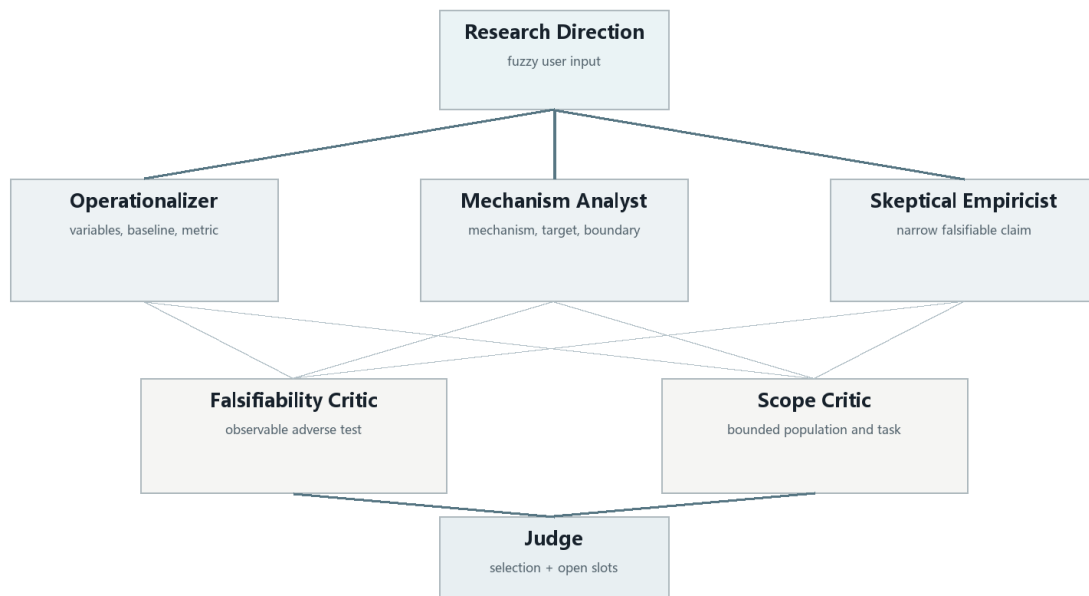


图 2 Multi-agent arena

三个 proposer 并行工作：Operationalizer 把方向变成变量、比较组和指标；Mechanism Analyst 明确机制、目标与边界；Skeptical Empiricist 提出最容易被不利结果证伪的窄主张。随后 Falsifiability Critic 与 Scope Critic 按 specificity、falsifiability、mechanism、scope、measurability、risk awareness 六维评分并提出公开修订。Judge 只读取结构化候选和批评，返回候选编号、最终主张、证伪实验和未解决歧义。

LLM 层使用最小 OpenAI-compatible chat client。模型和端点由环境变量或 UI 会话字段配置；本轮真实端点检测到 GPT-5 系列模型，因此客户端对模型名以 GPT-5 开头的请求不发送默认 temperature，以减少常见 request-shape 错误。这不等同于已经验证全部 GPT-5 接口能力。

最终 Web 使用 `llm_strict`: 三个 proposer、两个 critic 与 judge 必须全部成功并返回完整 JSON; 若请求超时、HTTP 失败或返回形状错误, 失败信息会被脱敏并终止本次运行。CLI、MCP 和自动化回归仍可显式选择确定性基线, 但 Web 不会把基线包装成真实 Agent 结果。

3.3 假设与证据链

Planner 生成原始主张、边界变体和失败变体, 并拆出四类通用假设: 目标设置中的效果真实、所需证据质量足够、评价指标有效、效果可跨数据集或领域泛化。每项假设都有 `support`、`contradict`、`limitation`、`null_result` 四条查询。Retriever 支持离线 fixture、arXiv 和 Semantic Scholar, 并按外部 ID 或标题去重。

Pipeline 把摘要句子映射到假设, 保留 `snippet offset` 和 `query kind`。保守判定规则包括: 未检索到材料时状态为 `unknown`; 只出现 `limitation` 不等于 `disproof`; 中性提及不计为负面证据; 支持和反向信号并存时为 `mixed`。Negative Evidence Miner 专门抽取 `failure mode`、`limitation` 和 `negative result`, 再由 Opportunity Builder 生成 `assumption gap` 或 `negative-evidence experiment slot`。

3.4 模块与接口

模块	主要输入	主要输出	功能
<code>core_claim</code>	<code>direction</code> , <code>LLM client</code>	<code>CoreClaimResult</code>	多角色候选、批评、裁决与降级
<code>planner</code>	<code>research direction</code>	<code>ClaimPlan</code>	主张变体、假设和四类查询
<code>retrievers</code>	<code>query</code> , <code>limit</code>	<code>Paper list</code>	离线与在线检索、去重、告警
<code>pipeline</code>	<code>planner</code> , <code>retriever</code>	<code>AnalysisReport</code>	证据映射、负面证据、机会点
<code>models</code>	<code>typed fields</code>	<code>JSON / Markdown</code>	数据约束、溯源与序列化
<code>service</code>	<code>JSON-compatible args</code>	<code>dict</code>	Web、CLI、MCP 统一边界
<code>mcp_server</code>	<code>tool call</code>	<code>structured payload</code>	五个可组合科研工具
<code>app.py</code>	<code>user input</code>	<code>interactive workflow</code>	同意门、Trace、导出与反馈

五个 MCP 工具分别为 `extract_core_claim`、`analyze_research_direction`、`build_evidence_queries`、`evaluate_claim_benchmark` 和 `get_demo_report`。它们使 ClaimScope 可以嵌入支持 MCP 的编辑器或上层智能体, 而不需要复制 UI 逻辑。

3.5 界面与报告设计

最终界面按论文审阅与代码评审工作台设计，而不是常见的“AI 仪表盘”。页面移除渐变、发光动画、彩色大卡片和拟人化思考文案，采用细分隔线、原生控件、文档式结果区和克制状态色。Web 支持中文与英文即时切换；运行期间实时显示 Proposer → Critic → Judge 的公开状态；结束后可分别展开六个角色，检查公开输入摘要、候选主张、六项评分、原因代码、修改建议与裁决结果。页面同时支持 390 px、768 px 与 1440 px 视口、键盘焦点及 prefers-reduced-motion。

实验报告参考 MiniMax Docx 的 CJK 排版和 OpenXML 规则：A4 页面、中文正文宋体 10.5 pt、标题微软雅黑、1.45 倍行距、自动目录字段、图表题注、首页独立页眉页脚与 Word/PDF 双格式验证。两个第三方 skill 均只作为设计与排版知识源，不参与科研结论生成。

4 实现过程记录

第一阶段实现最小 pipeline，验证“主张→假设→证据→机会点”数据结构是否连通。第二阶段把核心主张独立为可单测模块，并加入缺失槽位、结构得分和证伪测试。第三阶段实现三 proposer、两 critic、一 judge 的竞技场，限定并发数为三，加入 JSON schema 校验和 weighted fallback。第四阶段增加证据保守语义，修正“检索不到即反驳”和“限制即否认”两类危险错误。第五阶段增加 Web 双模式、公开 Trace、下载反馈、隐私同意门和移动端适配。第六阶段建立 ClaimBench、MCP、wheel clean-install 与 CI 矩阵。第七阶段引入项目级 UI/UX 与文档 skill，生成可追溯设计系统，再依据实际科研工作流做界面和课程报告的收敛优化。

最后验收时，先处理机器全局 Python 与项目隔离环境的依赖差异，随后创建 .venv 并执行 `pip install -e ".[web,mcp,dev]"`。加入严格 Agent 事件回调、双语 UI、真实检索同意门和浏览器契约后，同一解释器中全部 111 项测试通过。该过程说明安装态测试必须在声明的依赖环境中运行，也促使最终文档提供一条完整安装命令。

真实 API 验收分为失败与成功两轮。第一组凭据可列出 GPT 系列模型，但所有 chat completions 均返回 HTTP 503；系统如实显示 proposer 失败，没有回退成伪造的 Web 结果。第二组凭据列出 Claude Haiku、Sonnet 和 Opus 系列，`claude-sonnet-4-5` 最小请求成功。随后严格竞技场完成 6 个角色、3 个候选和 6 份批评产物，Judge 返回最终核心主张和证伪实验。两轮过程中均未记录 provider body、请求正文或密钥。

5 实验设计

5.1 实验问题

RQ1：启发式核心主张抽取能否在不同领域保持基本结构完整性？RQ2：异常输入、检索失败和外部模型失败时，系统能否继续返回可解释产物？RQ3：Web、CLI、MCP 与安装后的 wheel 是否共享一致行为？RQ4：系统是否明确区分合成演示数据、摘要级信号与科学结论？

5.2 数据与指标

ClaimBench 包含 27 个内部构造样例，覆盖 9 个领域，每个领域 3 个方向。评分由 slot coverage、declarative form、specificity、comparison language、measurable outcome、boundary condition、falsifiability、required concept coverage 与 overclaim penalty 组成。报告总分为 0 至 100。该基准用途是版本回归和错误定位，不是人工标注的科学有效性基准。

自动化测试分为 core claim、pipeline、retriever、benchmark、UI contract、repository health 和 MCP stdio 七类。额外验收包含 Python compileall、wheel 构建、仓库外 clean install、CLI、Streamlit health、MCP initialize/list_tools 与密钥扫描。

5.3 结果

指标	最终结果	解释
自动化测试	111 passed	隔离环境，包含严格 Agent、UI 与 MCP 测试
ClaimBench	27 cases	9 个领域，每领域 3 个样例
启发式均分	70.09 / 100	结构回归信号
MCP 工具	5	均可通过 stdio 列出
多智能体角色	6	3 proposer + 2 critic + 1 judge
workflow 阶段	7	从方向到机会点
UI 验证视口	3	390×844、768×1024 与 1440×900

典型输入为 “RAG can reliably reduce hallucination in LLM-generated answers” 。 Claude Sonnet 4.5 的严格运行生成 3 个候选；最终主张把效果收窄为 “在封闭领域事实问答中，当知识库召回率超过 0.8 且每次返回前三个相关段落时，RAG 至少降低 20% 的幻觉率” 。系统同时给出留出集证伪测试，并明确列出模型版本、测量协议、领域边界、显著性阈值和样本量等 5 类待补字段。结构分仅表示字段完整度，不表示该主张已经获得科学证实。

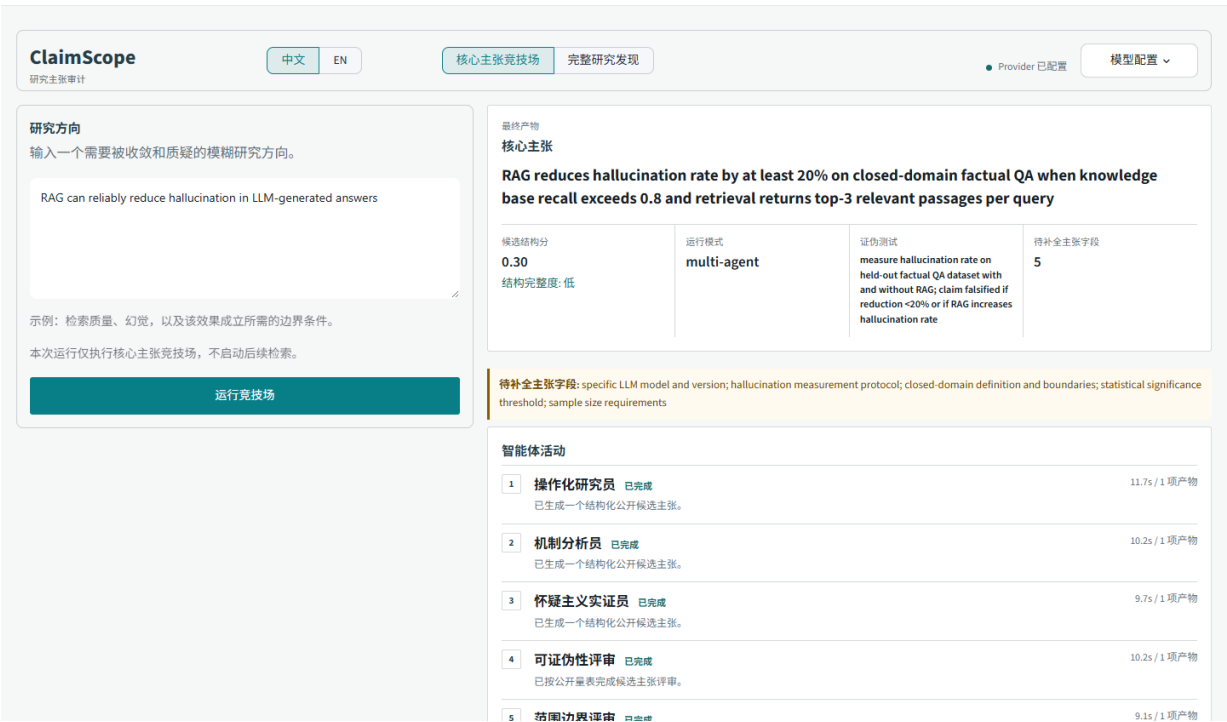


图 3 Claude Sonnet 4.5 严格六角色运行结果

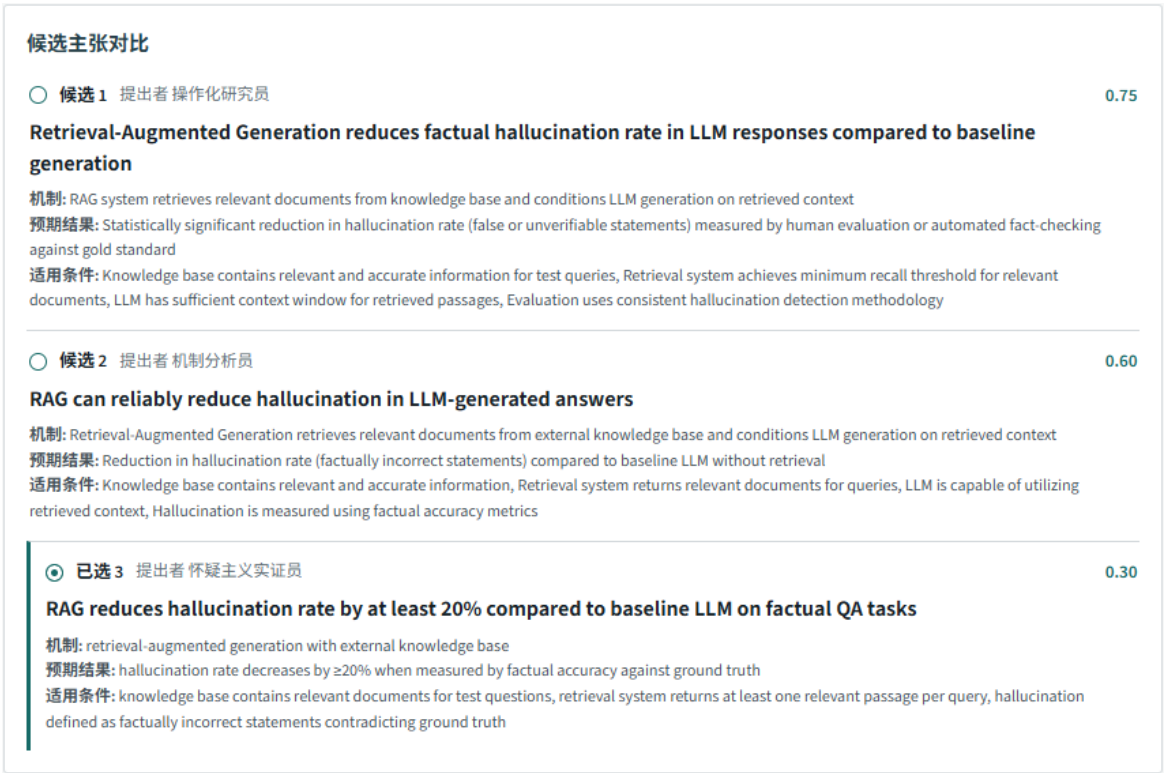


图 4 三个候选主张对比与 Judge 选择

系统在故障路径上也有可验证行为：不相关主张不会错误绑定 fixture；retrieval miss 保持 unknown；neutral mention 不会转成 negative evidence；单个 retriever 失败会变成脱敏 warning；严格 Web 模式下任何 proposer、critic 或 judge 缺失都会终止，不生成最终主张。协议级本地测试 Provider 仅用于验证成功布局，确认 7 个展开区、桌面与移动视口均无 Traceback；测试 Provider 在验收后已删除。

6 结果分析与反思

从课程评分角度，本项目的创新点不是“再做一个论文问答”，而是把文献探索的组织单位从 paper 或 question 变为 assumption。主动寻找负结果和限制条件可以减少 confirmation bias，机会点也由明确 gap 推导，而不是让模型凭空生成“看起来新颖”的 idea。MCP 工具进一步把流程变成可组合科研基础设施。

当前结果仍有四类限制。第一，ClaimBench 是内部构造且由规则评分，70.09 不能说明输出真实或有科研价值；下一步应由多名研究者标注可证伪性、边界清晰度和机会价值，并报告一致性。第二，离线论文为 synthetic fixtures，只用于界面和数据链演示。第三，在线检索以摘要和 snippet 为主，无法检查全文方法、统计显著性或复现实验。第四，LLM 模式依赖外部 provider，存在配额、模型参数兼容、超时与非确定性问题。

后续优化应优先加入带许可证约束的全文连接器、人工标注的 ClaimBench、single-LLM 与 multi-agent 的受控消融、调用成本和延迟统计、跨模型复测、引用句与 PDF 页码对齐，以及真实研究者 longitudinal study。系统输出应始终定位为“研究空间导航信号”，而非自动科学裁决。

7 总结

ClaimScope 完成了一个可运行、严格失败、可追溯的科研前置智能体系统。它把模糊方向逐步转成可证伪主张、隐藏假设、对抗查询、证据卡片、负面结果和有边界的实验机会；同时通过双语 Web、CLI、Markdown 和 MCP 暴露统一能力。最终 111 项测试、27 案例回归基准、真实 Claude 六角色运行和安装态验收说明工程链路完整。更重要的是，系统明确暴露未知项、外部 Provider 失败与摘要级证据的限制，避免把结构评分或失败回退误当成真实 Agent 成功。项目下一阶段的核心不是增加更多生成文本，而是建立更可靠的人类标注、全文证据与真实科研使用评估。

参考文献

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. <https://arxiv.org/abs/2005.11401>
- [2] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. <https://arxiv.org/abs/2210.03629>
- [3] Wadden D, Lin S, Lo K, et al. Fact or Fiction: Verifying Scientific Claims. EMNLP, 2020. <https://arxiv.org/abs/2004.14974>
- [4] Skarlinski M D, Cox S, Laurent J M, et al. Language agents achieve superhuman synthesis of scientific knowledge. 2024. <https://arxiv.org/abs/2409.13740>
- [5] MiniMax-AI. MiniMax Skills. <https://github.com/MiniMax-AI/skills>
- [6] NextLevelBuilder. UI/UX Pro Max Skill. <https://github.com/nextlevelbuilder/ui-ux-pro-max-skill>

附录 A 复现命令

```
python -m venv .venv
.venv\Scripts\python -m pip install -e ".[web,mcp,dev]"
.venv\Scripts\python -m pytest -q
.venv\Scripts\python -m claimscope.cli benchmark --engine heuristic
.venv\Scripts\streamlit run app.py
```

附录 B 提交检查

报告和视频文件名为“自然语言处理大作业-刘子谦-23354118”；视频不超过 1 分钟并含语音；提交前不得在画面、报告、Git 历史或附件中出现 API key；必须确认 Provider 可用后再录制真实 Agent 成功流程。